

Autómatos Finitos

Finite Automata

Nesta aula, alguns slides serão apresentados em português e outros em inglês.

Esta opção visa apoiar simultaneamente estudantes portugueses e estudantes internacionais, bem como promover a familiarização com a terminologia científica utilizada na literatura da área.

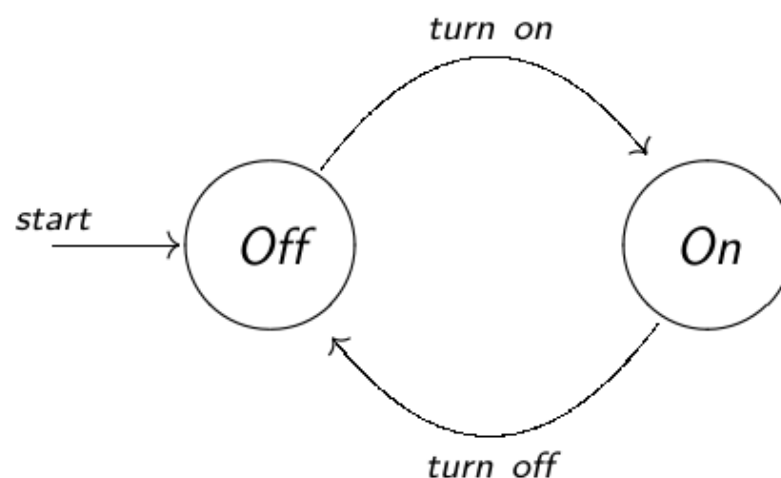
In this lecture, some slides will be presented in Portuguese and others in English.

This approach aims to support both Portuguese and international students while fostering familiarity with the scientific terminology commonly used in the literature.

What is a Finite Automaton (FA)?

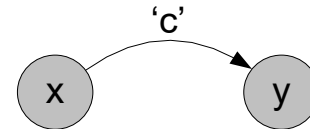
Definition of FA

A **finite automaton** or **state machine**, is a formal way of representing any process composed by a (finite) set of states, and transitions between those states.



Autómatos Finitos

- As expressões regulares são convenientes para especificar os *tokens* de uma linguagem. Que formalismo usar para que possa ser facilmente implementado num programa?
- Modelo matemático conhecido por autómato finito (AF), definido por:
 - ◆ o alfabeto de entrada Σ
 - ◆ um conjunto de estados K
 - ◆ um conjunto de transições entre estados para um caracter de Σ (ou ϵ) do tipo “do estado x transita-se para o estado y com a entrada ‘c’ “
 - ◆ um conjunto de estados de aceitação ou finais
 - ◆ um estado inicial S
- Um autómato finito pode ser representado por uma tabela ou, mais frequentemente por um grafo, em que os nós são estados e os ramos transições, etiquetados com os caracteres que determinam essa transição; os estados inicial e finais deverão estar marcados



Formal definition of a Finite Automaton

A Finite Automaton (FA) is represented by a tuple (a set of five elements):

$$A = (S, \Sigma, s_0, F, \delta)$$

- S a finite (non-empty) set of states;
- Σ is the input alphabet, a finite (non-empty) set of input symbols;
- s_0 is the initial state, an element of S ($s_0 \in S$);
- F is the set of final states. F is a subset of S ($F \subset S$);
- δ is the transition function, having as arguments:
 - one state
 - one input symbol

and returns the following state (it may be the same):

$$\delta : s \times a \rightarrow T$$

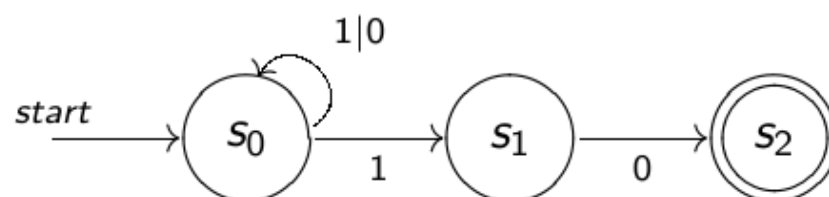
or

$$\delta(s, a) = T, \text{ with } s \in S \wedge a \in \Sigma \wedge T \subseteq S$$

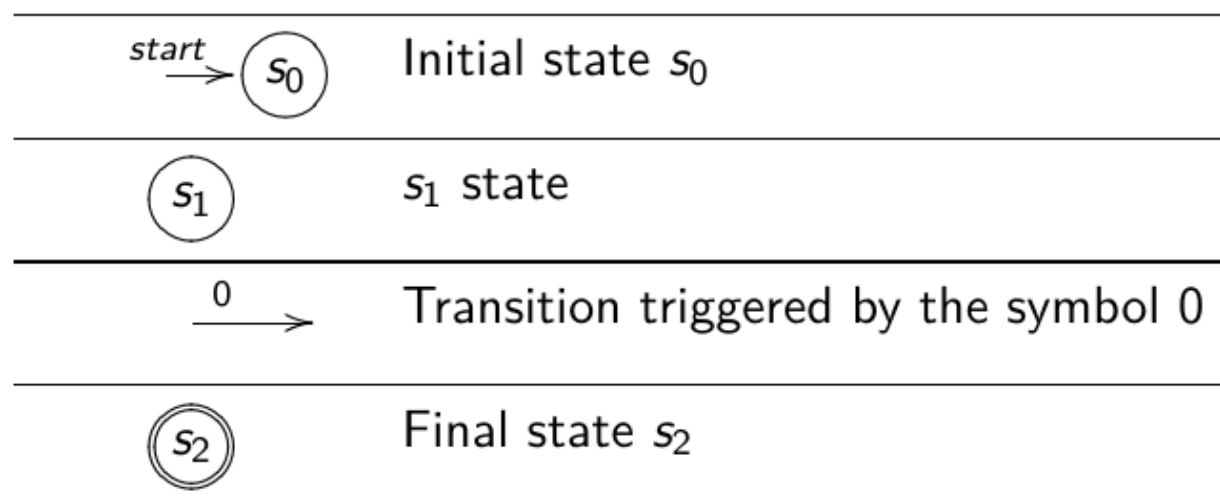
Graphical representation of a Finite Automaton

Consider a finite automaton able to process binary numbers ending in “10”, equivalent to the regular expression “ $(0|1)^*10$ ”

The Finite Automaton can be represented graphically:



in a way:



Formal representation of a finite automaton and transition table

Consider a finite automaton able to process binary numbers ending in “**10**”, equivalent to the regular expression “**(0|1)*10**”

$$A = (\{s_0, s_1, s_2\}, \{0, 1\}, s_0, \{s_2\}, \delta)$$

where,

$$\delta(s_0, 0) = \{s_0\}$$

$$\delta(s_0, 1) = \{s_0, s_1\}$$

$$\delta(s_1, 0) = \{s_2\}$$

A FA can also be represented by a transition table:

	0	1
$\rightarrow s_0$	$\{s_0\}$	$\{s_0, s_1\}$
s_1	$\{s_2\}$	$\emptyset$
$*s_2$	$\emptyset$	$\emptyset$

Aceitação de AFN's

- Uma *string* $x_1 \dots x_n$ é aceite por um AFN se existir um caminho começando no estado inicial e terminando num estado final, cujas etiquetas dos ramos percorridos coincidam, por ordem, com os caracteres x_1 a x_n , com a possível inclusão, em qualquer local, de qualquer número de transições ϵ
- Os autómatos finitos (determinísticos e não determinísticos) definem linguagens, que são exactamente o conjunto de *strings* aceites pelo autómato

Validation of a phrase/string using a FA

The processing of a string by a FA is done in this way:

- The automaton “is placed” on the initial state, before reading the first character of the string
- If for each character there is a valid transition, the automaton is placed on the specified state, and input is advanced to the next character; If **there is no transition function**: the automaton stops and the **string is rejected**;
- If when the input string ends the automaton is in a **final** state: the automaton stops and the **string is accepted**;
- If when the input string ends the automaton is in a **non final** state: the automaton stops and the **string is rejected**;

Automata Classification

An automaton is said to be a **deterministic** finite automaton (**DFA**) if, in every state, for a single symbol, there is only a transition for a single state *i.e.*,

$$\forall s_1, s_2, s_3, a : \delta(s_1, a, s_2) \in \delta \wedge \delta(s_1, a, s_3) \in \delta \rightarrow s_2 = s_3$$

$$\text{Nota : } \delta(s_1, a, s_2) \Leftrightarrow \delta(s_1, a) = \{s_2\}$$

If that is not true, the automaton is said to be **non deterministic** (**NFA**).

One can have also finite automata **with empty transitions** (**FA ϵ**) where there are transitions that don't use any symbol from Σ . They can be used for the conversion of Regular Expressions to Finite Automata. DFA versus NFA:

- DFA are faster (computation time) than NFA;
- NFA have a smaller memory occupation (less states) than DFA;
- A DFA can be considered as particular case of an NFA where in all states K for any input symbol x , there is only a transition out of K , for a unique value of x .

AFN's e ER

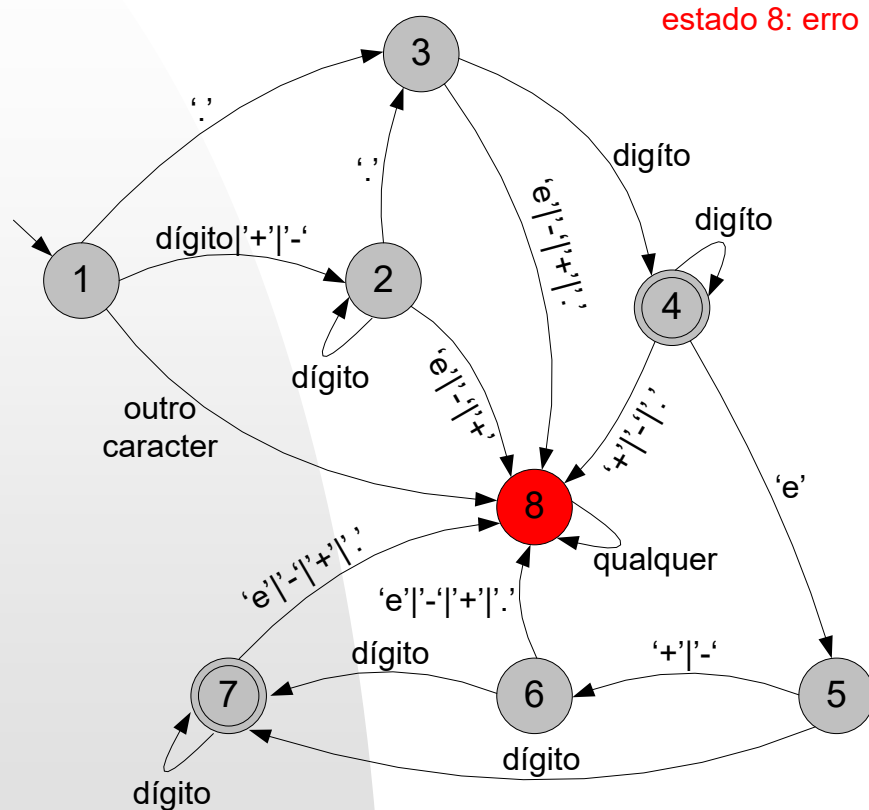
- Existe “equivalência” entre expressões regulares (ER), autómatos finitos determinísticos (AFD) e autómatos finitos não determinísticos (AFN)
 - ◆ Dada uma expressão regular é sempre possível construir um AFN que aceita exactamente a mesma linguagem
 - ◆ Dado um AFN é sempre possível construir um AFD que aceita exactamente a mesma linguagem – algoritmo da “construção dos subconjuntos”
 - ◆ Dado um AFD é sempre possível construir uma ER que aceita exactamente a mesma linguagem

Exemplos

- Sequence with a suffix of 1
- Sequence with a prefix of 1
- Sequence that starts with 1 and ends in 0
- Sequence that contains two 1s
- Sequence that contains the factor 01
- Sequence that starts and ends with the same digit

Autómatos finitos (exemplos)

$\langle \text{número real} \rangle = ('+'|-) ? \langle \text{dígito} \rangle^* . \langle \text{dígito} \rangle^+ (e ('+'|-) ? \langle \text{dígito} \rangle^+) ?$

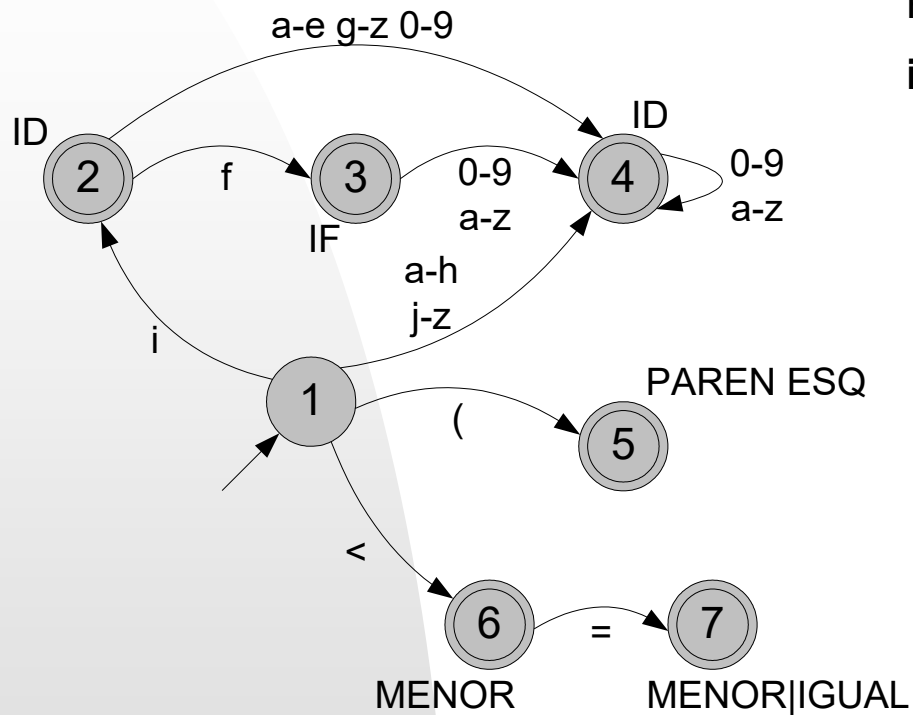


```

function realnum: boolean;
var  entrada: char;
     estado: integer;
begin
    estado:=1; read(entrada);
    while (entrada>='0' and entrada<='9') or
           (entrada='+' or entrada='-') or
           (entrada='e') or (entrada='.') do
    begin
        case estado of
            1: case entrada of
                  '0','1',...,'9': estado=2;
                  '-', '+': estado=2;
                  '.': estado=3;
                  otherwise: estado=8;
                end;
            2: ...
            ...
        end;
        read(entrada);
    end;
    realnum:=(estado=4) or (estado=7)
end
  
```

Autómatos finitos (exemplos)

Autómato finito que reconhece os *tokens*: `if id ( < <=`



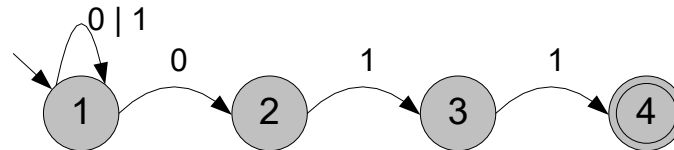
Para reconhecer uma entrada como:

`if ( a <= ...`

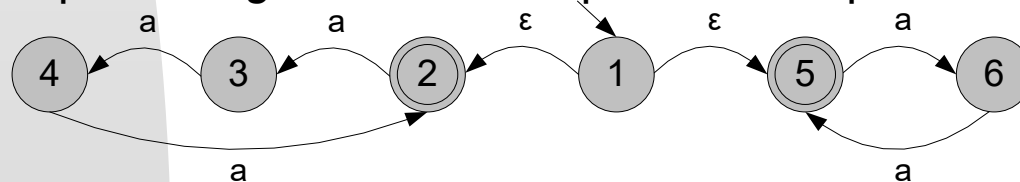
- seguem-se as transições de estado começando no estado inicial até não se conseguir ir mais além (máxima *substring*);
- nessa altura se estivermos num estado final reconhece-se o *token* respectivo;
- se não estivermos num estado final recuamos, carácter a carácter, até passarmos por um estado final.

Autómatos finitos não-determinísticos (AFN)

- Os autómatos anteriores são determinísticos (as transições entre estados fazem-se apenas com caracteres do alfabeto, e todas as transições que saem de um estado estão etiquetadas com caracteres diferentes)
- Certas *strings* são mais fáceis de especificar com um autómato não determinístico
 - ◆ Exemplo: Todas as *strings* no alfabeto $\{0, 1\}$ terminadas por 0 1 1



- Os autómatos finitos não determinísticos podem ter transições de estado sem consumirem qualquer entrada - as chamadas transições ϵ
 - ◆ Exemplo: *Strings* constituídas por um múltiplo de 2 ou 3 a's

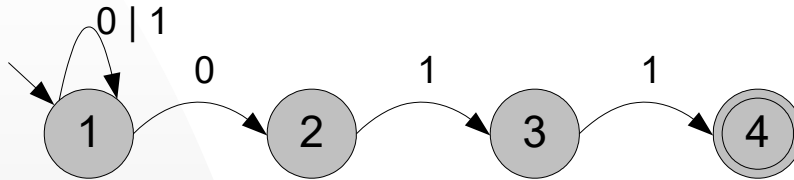


Equivalência entre AFN e AFD

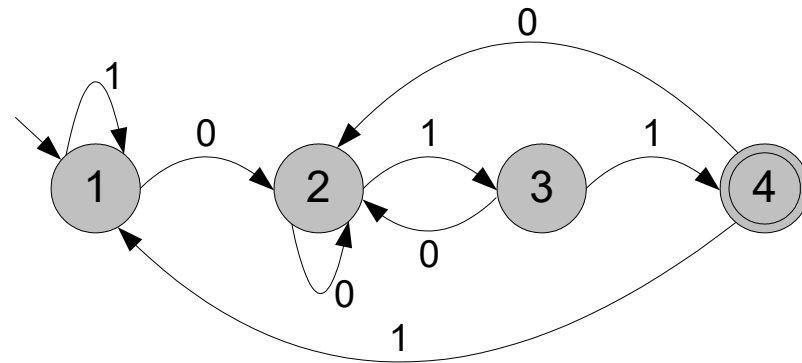
(exemplo)

Exemplo: Todas as *strings* no alfabeto $\{0, 1\}$ terminadas por 0 1 1

AFN



AFD

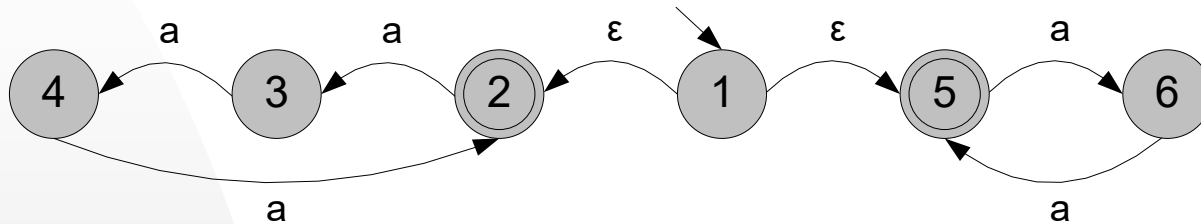


Equivalência entre AFN e AFD

(exemplo)

Exemplo: *Strings* constituídas por um múltiplo de 2 ou 3 a's

AFN



AFD

